AI Study Assistant – Intelligent Personalized Learning & Assessment System

Author / Student Name: Syed Haider Ali Shah

Course: SCD

Domain: Artificial Intelligence, Retrieval-Augmented Generation (RAG), and Educational Technology

1. Introduction / Background

In modern higher education, students are frequently overwhelmed by vast volumes of unstructured course materials, including PDF lecture notes, slides, recommended textbooks, and assignment briefs. While generative AI models (such as ChatGPT and Gemini) offer general tutoring capabilities, they suffer from critical limitations in an academic context: they rely on parametric general knowledge rather than authorized course syllabi, often produce hallucinations, and lack continuous tracking of a student's conceptual mastery.

The **AI Study Assistant** addresses this gap by functioning as an intelligent, course-specific personalized learning and assessment system. By combining Retrieval-Augmented Generation (RAG) with adaptive assessment and learning analytics, the system grounds all AI interactions strictly in the student's authorized course repository.

2. Problem Statement

University students lack a unified, reliable, and context-aware digital study companion capable of processing authorized course materials to deliver verified explanations, targeted practice questions, and precise weak-topic analytics. Existing tools either operate outside course boundaries with high hallucination risks, or lack automated formative assessment capabilities.

3. Existing Problems

- **Information Overload:** Managing hundreds of pages of disparate PDF slides, notes, and textbook chapters without centralized semantic indexing.
- **AI Hallucinations:** General-purpose chatbots generate plausible-sounding but factually incorrect explanations untethered from course syllabi.

- **Lack of Personalization:** Static reading materials do not adapt difficulty or provide targeted diagnostic quizzes based on individual student performance.
- **Absence of Source Attribution:** Students cannot easily verify where in their lecture notes or readings an AI-generated explanation originated.

4. Proposed Solution

We propose developing an **AI Study Assistant** built around a secure document processing and Retrieval-Augmented Generation (RAG) architecture. The system ingests authorized course materials, creates semantic vector embeddings, and provides an interactive workspace where students can query documents, receive citation-backed explanations, generate practice quizzes, and track conceptual mastery over time.

5. Motivation

The motivation stems from the need to harness Artificial Intelligence ethically and reliably within academic environments. By enforcing source grounding and transparency, this project demonstrates how machine learning can enhance student comprehension and reduce academic friction without compromising academic integrity.

6. Objectives

- Develop a secure document ingestion and text-chunking pipeline for PDF lecture notes, slides, and textbooks.
- Implement an efficient RAG retrieval mechanism using vector embeddings to ground LLM answers in course materials.
- Build an automated question generation engine (MCQs and short questions) with adaptive difficulty scaling.
- Construct a student progress dashboard and analytics engine for weak-topic identification and revision recommendations.
- Evaluate system accuracy, latency, and hallucination reduction against benchmark datasets.

7. Scope

In-Scope (Semester Project): User authentication, course workspace management, PDF text extraction, semantic vector search, RAG-powered Q&A with source citations, automated MCQ generation, practice test grading, and student performance dashboard.

Out-of-Scope (Future Enhancements): Multi-institution enterprise administration, real-time peer collaborative study rooms, live video lecture transcription, and native mobile apps (the semester project will focus on a responsive web application).

8. Target Users / Stakeholders

- **Students:** Primary users seeking personalized study support, practice tests, and concept clarification.
- **Instructors / Teaching Assistants:** Secondary stakeholders who can monitor class-wide weak topics via analytics dashboards (optional module).
- **University Administrators:** Overseeing platform data privacy and system security compliance.

9. Core Functional Requirements

- **FR-01:** The system shall authenticate users securely using JWT-based role authentication (Student / Instructor).
- **FR-02:** The system shall allow users to upload, store, and manage course-specific PDF documents and lecture notes.
- **FR-03:** The system shall extract text from uploaded PDFs, chunk text segments, and generate vector embeddings stored in a vector database.
- **FR-04:** The system shall process student queries using RAG to retrieve relevant course text and generate citation-backed answers.
- **FR-05:** The system shall automatically generate multiple-choice questions (MCQs) and short quizzes from selected course chapters.
- **FR-06:** The system shall evaluate student quiz submissions, compute performance scores, and update topic mastery metrics.

10. Non-Functional Requirements

- **NFR-01 (Performance):** RAG-based query responses shall be generated within 3 seconds under normal load.
- **NFR-02 (Security):** All user passwords shall be hashed using bcrypt, and API endpoints shall enforce strict role-based access control.
- **NFR-03 (Reliability):** The system shall successfully retrieve source citations for at least 95% of factual course queries.
- **NFR-04 (Usability):** The web interface shall be responsive and accessible across modern desktop and tablet browsers.

11. Major System Modules

- **Authentication & User Management Module:** Handles user registration, login, and access control.
- **Document Processing & Ingestion Module:** Handles PDF parsing, cleaning, text chunking, and embedding generation.
- **Knowledge Retrieval & RAG Module:** Performs semantic similarity search and prompt construction with source citations.
- **Assessment & Quiz Engine Module:** Generates adaptive questions and evaluates student test responses.
- **Analytics & Progress Dashboard Module:** Visualizes student mastery scores and identifies weak topics.

12. AI / Intelligent Features

- **Source-Grounded RAG Q&A:** Answers queries strictly using retrieved document context.
- **Automatic Question Generation:** Synthesizes pedagogically sound MCQs from course text.
- **Weak-Topic Diagnostic Engine:** Analyzes quiz errors to pinpoint conceptual gaps.
- **Adaptive Difficulty Scaling:** Adjusts subsequent quiz difficulty based on historical performance.

13. Key Innovative Features

Unlike generic AI chatbots, the system enforces a **strict document-grounding threshold**. If a query falls outside the uploaded course workspace, the system refuses to guess and prompts the user to upload relevant material, drastically minimizing hallucination in academic settings.

14. Complex Computing Problem (CCP) Justification

To satisfy HEC/NCEAC requirements, the project demonstrates complexity across multiple dimensions rather than being a basic CRUD application:

CCP Characteristic	How the AI Study Assistant Demonstrates It	Concrete Example
Depth of Knowledge	Requires advanced understanding of NLP vector embeddings, cosine similarity search, chunking strategies, and prompt engineering.	Configuring recursive text splitters and tuning chunk overlap to preserve academic context across page breaks.
Complex Analysis	Parsing unstructured multi-format PDFs, extracting tables/equations, and mapping quiz performance to hierarchical course topics.	Analyzing student error patterns across 5 chapters to isolate specific weak sub-concepts.
Conflicting Requirements	Balancing low generation latency against high contextual retrieval accuracy and strict token budget limits.	Optimizing top-k retrieval parameters to maximize context relevance while avoiding LLM context window overflow.
Interdependence	Tight coupling between document preprocessing, vector database indexing, LLM API calls, and relational analytics storage.	Synchronizing asynchronous document parsing tasks with real-time frontend WebSocket/API state updates.
Significant Impact	Directly mitigates AI hallucination in education and provides measurable diagnostic insights for student learning.	Providing exact page number citations for every generated study answer, ensuring academic traceability.

Feature	ChatGPT / Gemini	Khanmigo	Google NotebookLM	Proposed AI Study Assistant
Course Material Grounding	No (General Web)	Curated Curriculum	Yes (User Uploads)	**Yes (Course Workspace)**
Adaptive Assessment	Manual prompt	Socratic Dialogue	None	**Automated MCQs & Quizzes**
Learning Analytics	None	Teacher View	None	**Student Progress Dashboard**
Source Citations	Inconsistent	Curriculum Links	Document Citations	**Exact Page/Slide Citations**

15. Benchmarking with Existing Systems / Literature

We benchmark our system against leading educational and AI tools to highlight the specific research gap addressed:

Identified Gap: Existing tools lack an integrated combination of localized academic document RAG grounding, automated formative assessment generation, and student weak-topic analytics in a unified semester project architecture.

16. Proposed System Architecture

The system architecture follows a layered client-server design:

`Student/Teacher UI (React) → REST API / FastApi Gateway → Auth & Session Layer → Document Processing & Chunking → Vector Store (ChromaDB) & Relational DB (PostgreSQL) → RAG Retrieval & LLM Engine → Assessment & Analytics Engine.`

17. Tools & Technologies

- **Frontend:** React.js, Tailwind CSS, Lucide Icons.
- **Backend:** Python FastAPI, LangChain / LlamaIndex.
- **Database:** PostgreSQL (Relational data), ChromaDB / pgvector (Vector embeddings).
- **AI / LLM API:** OpenAI GPT-4o-mini / Open-source Llama-3 via Groq/HuggingFace API.
- **Deployment:** Docker, Docker Compose, local development server / cloud hosting.

18. Expected Outcomes and Impact

The project will deliver a fully functional, tested prototype of the AI Study Assistant. Expected impact includes a 40% reduction in time spent searching through lecture materials, verified elimination of ungrounded hallucinations for course queries, and actionable diagnostic analytics for students.

19. Risks and Mitigation

- **Risk:** LLM API rate limits or cost overruns.
  Mitigation: Implement local embedding caching and utilize efficient, low-cost model tiers (e.g., GPT-4o-mini).
- **Risk:** Poor text extraction quality from scanned PDF slides.
  Mitigation: Integrate fallback OCR preprocessing libraries (PyTesseract) for image-based slides.

20. Development Methodology

An Agile iterative development methodology will be used across a 14-week semester, divided into 3 major milestones: Architecture & Ingestion (Weeks 1–4), RAG & Assessment Engine (Weeks 5–9), and Analytics, Testing & Deployment (Weeks 10–14).

21. Project Timeline (14-Week Semester)

- **Weeks 1–4:** Requirements analysis, database schema design, wireframes, and document upload pipeline.
- **Weeks 5–9:** PDF text extraction, vector embedding, RAG query engine, and quiz generation module.

- **Weeks 10–14:** Student progress dashboard, weak-topic analytics, end-to-end testing, and final documentation.

22. Conclusion

The AI Study Assistant project provides a robust, academically grounded solution to information overload and generic AI hallucination in higher education. By satisfying stringent HEC/NCEAC Complex Computing Problem criteria, this project bridges modern generative AI with rigorous academic course materials.